

Тема 10. ЦЕНА НА СОФТУЕРА

дисциплина: Технология на Софтуерното Производство
лектор: доц. В.Божикова

Съдържание

- 1. Необходимост, история и ползи***
- 2. Критерии***
- 3. Моделът на Боем СОСОМО***
- 4. Метод на функционалните точки***
- 5. Други модели***
- 6. Оценяване при обектна ориентираност***

1. Необходимост, история и ползи

Необходимост от оценка на разходите за разработка (Software Cost Estimation)

- Всеки ръководител на софтуерен проект би желал във възможно най-ранен момент от жизнения цикъл на разработвания програмен продукт да знае колкото се може по-точно какви разходи ще трябва да се направят до завършването му и колко ще продължи целият процес [1],[2].

История...

- **SDC**, 1965 г. - първият метод за оценяване разходите по произвеждането на даден програмен продукт на Systems Development Corporation [1÷5].
- **COCOMO (COst COnstructive MOdel)**, 1981 г. - предложен от Боем [1÷6]. Тази разработка се смята за фундаментална — от една страна, тя намира реално приложение в практиката, а от друга — поставя на здрави основи последващите изследвания по цената и по-общо — по икономиката на разработването на софтуер.

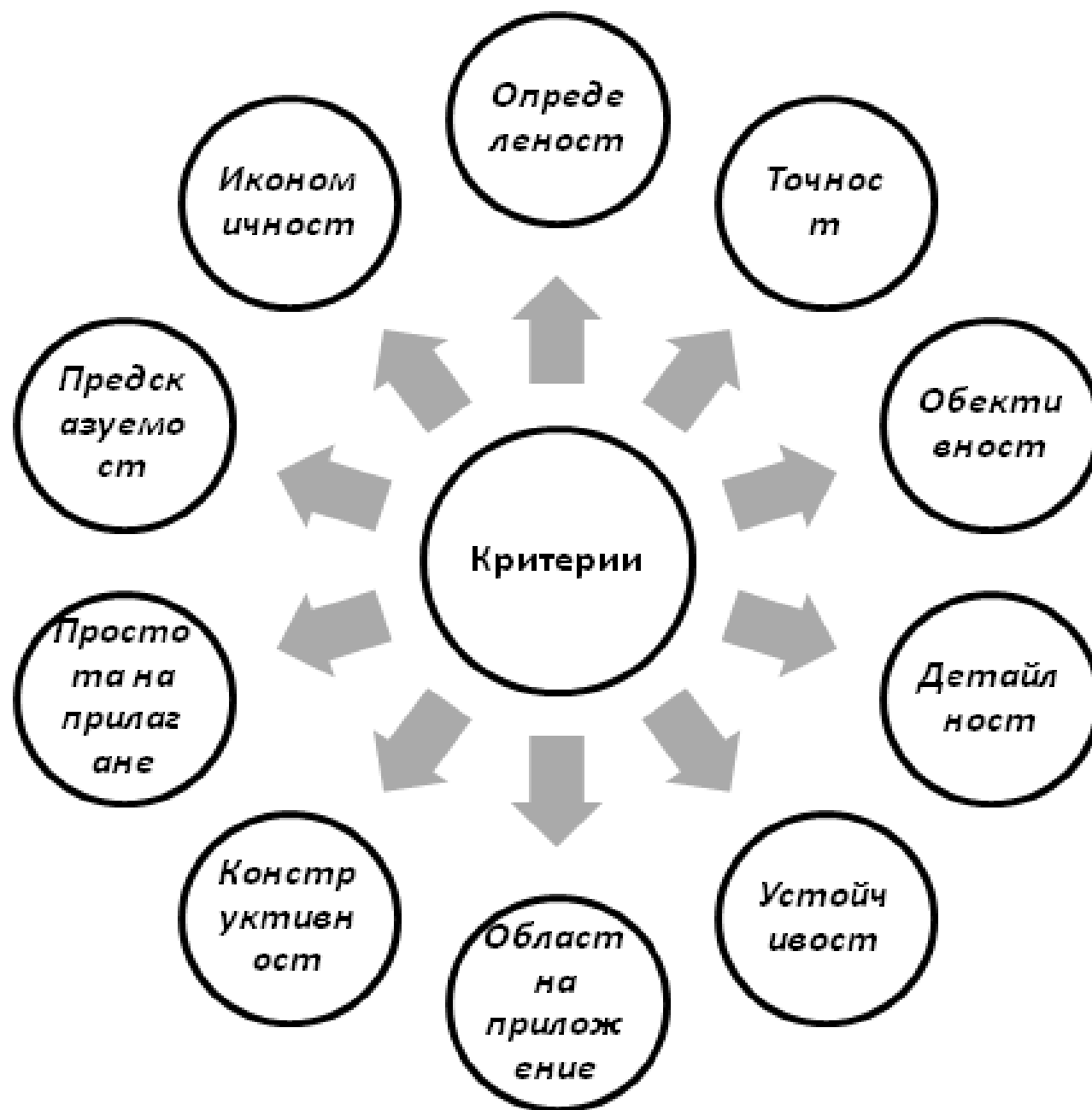
Ползи

Процесът на намиране цената на разработване води до резултати, полезни и в други насоки [2]:

- ***избор на проект за реализация*** — очевидно един от решаващите фактори при такъв избор е цената на бъдещата разработка;
- ***определяне състава на колектива разработчик***, което също е в пряка зависимост от установената цена и, обратно — изменяйки състава (по брой, квалификация, опит и др.), можем пряко да влияем върху цената на разработката;
- ***определяне на маркетинговата политика*** по отношение разработвания софтуер
- ***оценяване работата на членовете на колектива*** — това може да става на основата на сравнение на предварително получените оценки, с крайните резултати от работата.

2. Критерии

- Боем [4÷5], авторът на COSOMO формулира множество от критерии, които следва да удовлетворява всеки модел (метод) за установяване цената на разработване на софтуерни продукти.



Критериите, формулирани от Боем:

2.1. Определеност

- Има се предвид възможността за **количествено определяне** на **началните данни и крайните резултати** за даден модел (метод) за установяване на цената. Тази възможност е предпоставка за успешното прилагане на модела.
- Смята се, че една от причините моделът СОСОМО да е получил широка известност и приложение е тъкмо отличното спазване на този критерий.

Критериите, формулирани от Боем:

2.2. Точност

- Има се предвид **съответствието между предсказаната от модела цена на разработване и реално получената се цена.**
- По отношение на точността - **няма метод**, който да дава една и съща точност за всички класове програмни продукти, т.е **няма универсални модели и методи**. Практиката е доказала, че колкото и да се опитват авторите да направят модела и метода си **универсален**, по правило **той е точен за определен клас програмни продукти**. При всеки опит за прилагането му извън този клас, точността на резултатите рязко се влошава и тогава се налага настройка — промяна на базовите уравнения, добавяне на нови фактори, премахване на нерелевантни такива.

Критериите, формулирани от Боем:

2.3. Обективност

- Както и при други оценяващи процедури, важен критерий е **постигането на възможно най-голяма обективност, т.е максимално избягване на субективния фактор при оценяването.**
- Затова, навсякъде където се очаква лична оценка от страна на експерти се изготвят:
 - предварително указания,
 - въвеждат се тегла, определянето на стойностите на които се формализира в рамките на възможното,
 - предоставят се еталони и др.... с цел – да се минимизира субективния фактор.

Критериите, формулирани от Боем:

2.4. Детайлност

- По принцип, колкото един модел е по-подробен, толкова той е по-адекватен на реалните обекти и процеси и следва основаният на него метод да даде по-точни резултати.
- Но, по-дълбоката детайлност изисква както повече ресурси (време, хора, пари) в етапа на разработване на модела, така и при прилагането му. Затова се търси компромис между цената на извършваната оценка и желаната степен на точност. В споменатия вече СОСОМО този проблем е решен с помощта на 3 предлагани версии на модела, с различно ниво на детайлност:
 - базова,
 - междинна и
 - детайлна.

Критериите, формулирани от Боем:

2.5. *Устойчивост*

- Този критерий касае **границите за успешно приложение на даден метод.**
- Така например, моделът DOTY се неточен при оценка на продукти с над 10 000 реда първичен код , тогава започват да се наблюдават силно разминаване на реалните стойности с изчисленията.
- COSOMO се счита за неустойчив при оценяване на малки проекти — до 2 000 реда първичен код.
- Една от причините за това явление е, че всички модели са създавани на основата на изследване на големи и много големи проекти и уравненията, чрез които се пресмятат оценките, се базират на тях.

Критериите, формулирани от Боем:

2.6. Област на приложение

- Както казахме, няма универсален модел за определяне цената и продължителността на разработване на софтуерен продукт. Следователно, за всеки модел трябва да може ясно да се определи областта на приложение.
- Така например **PRICE S** е предназначен за оценка на аерокосмически софтуер, а методът на функционалните точки работи най-добре за информационни системи и бизнес приложения.
- Областта на приложение на някои модели е по-широка благодарение на възможността, определени коефициенти в уравненията за оценка да бъдат променяни. Отново пример е COSOMO, както ще се види по-нататък.

Критериите, формулирани от Боем:

2.7. Конструктивност

- Конструктивността се свързва с **възможността да бъдат анализирани и обяснени получените резултати в резултат на прилагане на даден модел за оценка.**
- Както се подчерта, в някои случаи, даден модел дава по-ниски | по-високи резултати от реално получените. Конструктивността се свързва с възможността получените отклонения да бъдат обяснени.
- Целта на този критерий е конструктивна — **подобряване на модела.**

Критериите, формулирани от Боем:

2.8. Простота на прилагане

- Този критерий определя **степената на трудност** на
 - **разбирането и получаването на входните данни**, както и
 - **изпълнение на процедурите по оценяване.**

2.9. Предсказуемост

- Този критерий **засяга проблема за практическото използване на моделите**. Стремешът на всеки потребител е да получи възможно най-точна оценка за цената, **колкото се може по-рано**, а това не винаги е възможно.
- Например, моделът COSOMO се основава на броя редове първичен код, а това не е лесно предсказуем параметър, т. е. дори и много голям опит не може да гарантира точното му определяне в началната фаза на жизнения цикъл на софтуерния продукт.

Критериите, формулирани от Боем:

2.10. Икономичност

- Това изискване е очевидно, но и то като много от вече изброените има компромисен характер.
- От една страна, желателно е входните данни за оценката да бъдат по-малко на брой и лесно измерими, от друга — да не са прекалено малко, така че да доведат до фатална неточност на крайните резултати. С други думи — желателна е икономичност, но не и на всяка цена.
- При отделни модели е забелязано, че когато се преминава от една област на приложение към друга, някои от факторите се оказват в новата област толкова взаимосвързани, че отпадането на някои от тях става възможно. По този начин се повишава икономичността на модела.

3. Моделът на Боем COSOMO

3.1. Цели и развитие <https://en.wikipedia.org/wiki/COCOMO>

- COSOMO произлиза от Constructive Cost Model.
- Основната му цел е, за всеки планиран софтуерен проект да се оцени:
 - **цената (трудоемкостта в ЧМ - брой човекомесеца) и**
 - **срокът на разработване.**
- Основополагащите му идея е използването **броя редове първичен код.**
- Моделът е търпи развитие, тоест първоначално предложеният модел е усъвършенстван в различни направления чрез:
 - а. въвеждане на 3 нива на подробност (има базова, междинна и детайлна версия на CoCoMo),
 - б. отличаване на 3 типа софтуерни проекти (разпространен, полунезависим и вграден),
 - с. въвеждане на различни коефициенти при определяне на трудоемкостта (във връзка с б.) и отчитане на допълнителни атрибути, със съответни рейтинги (във връзка с а.).
 - д. създаване на **COSOMO 2.0** – с нов метод за оценка на размера на софтуера

3. Моделът на Боем COSOMO

3.2. Същност на модела [1],[2]:

- За оценяване на трудоемкостта на даден софтуерен проект се прилага формулата:

$$\text{ЧМ} = 2.4 \times \text{ХРПК}^{1.05},$$

- където ЧМ означава брой човекомесеца
 - ХРПК означава хиляди реда първичен код, тоест KLOC (Kilo Lines of Code).
- Оттук може да се изчисли производителността на труда:

$$\text{Производителност на труда} = \text{ХРПК} / \text{ЧМ}$$

- За оценяване продължителността (В, т.е времето) на разработване на софтуерния проект формулата е:

$$\text{В} = 2.5 \times \text{ЧМ}^{0.38},$$
 където В е срокът на разработване в месеци.

- Оттук може да се изчисли колко хора са нужни за разработка на проекта:

$$\text{Екип} = \text{ЧМ} / \text{В},$$
 където Екип е броя хора, необходими за разработка на проекта т.е. „човека“.

Формулите се прилагат при следните *предположения:*

- а) редовете първичен код (т. е. тези, които се пишат на някакъв език за програмиране LOC):
 - се броят **без коментарните редове**;
 - се отнасят за **крайния продукт** (а не за междинни версии);
 - не включват използваните стандартни програми;
- б) **включват се** само фазите **проектиране, програмиране и оценка**, включително усилията по управлението и документирането по време на тези фази;
- в) **не се включват обучението, планирането и инсталирането на софтуера при потребителя**;
- г) **включва се трудът на проектантите и програмистите**, но не и този на компютърните оператори, висшите ръководители и секретарките;

- д) смята се, че **един човекомесец е от 19 дни**, или 152 часа;
- е) предполага се, че продуктът е разработен съгласно спецификацията на изискванията (SRS) и никакви сериозни промени не са направени след това, тоест не се включва еволюция;
- ж) двете страни — потребителят и разработчикът — се предполага, че са добросъвестни през цялото време.

3.3. Пример

- Да предположим, че в резултат на предварителна експертиза бъдещият софтуерен продукт се оценява на **32 000** реда първичен код.
- Като приложим двете формули, ще получим следните числови резултати:

$$\text{ЧМ} = 2.4 \times 32^{105} = 91 \text{ (човекомесеца)}$$

$$\text{В} = 2.5 \times 91^{038} = 14 \text{ (месеца)}$$

- От тези базови резултати могат да се получат още две важни характеристики:

$$\text{Производителност: } 32\,000 / 91 = 352 \text{ реда първичен код за човекотомесеца}$$

$$\text{Екип: } 91 / 14 = 6.5 \text{ човека.}$$

- Тълкуванието на последната бройка е: **най-малко 7 човека** от екипа ще бъдат заети с проекта през всичките 14 месеца на разработката.

Размер на проекта и производителност

- Правени са изследвания за това, как се изменя производителността с промяната на размера на проекта.
- Боем предлага едно условно разделяне на проектите на **малки**, **междинни**, **средни** и **големи** и следните данни:

Тип проект	ХРПК	ЧМ	В	Екуп: брой	Производителност
Малък	2	5,0	4,6	1,1	400
Междинен	8	21,3	8,0	2,7	376
Среден	32	91,0	14,0	6,5	352
Голям	128	392,0	24,0	16,0	325

- Намаляването на производителността с нарастването на размера на проекта е естествено, поради увеличаването на броя на взаимодействията между растящия брой членове на колектива разработчик, всяко от които изисква допълнителни усилия и отнема от времето за създаване на първичния код.

3.4. Усъвършенстване на модела

а) въвеждане на 3 типа софтуерни проекти

- Първото усъвършенстване на дотук изложения **базов модел** е разграничаването на 3 типа софтуерни проекти:
 - *разпространен (organic),*
 - *полунезависим (semidetached) и*
 - *вграден (embedded).*
- За всеки от тях формулата за изчисляване на трудоемкостта на даден софтуерен проект е различна.

- В таблицата (таблица 2) тук всеки тип се характеризира кратко, илюстрира се с примери и му се съпоставя предложената от Боем формула.

Тип	Характеристика	Примери	Формула
Разпространен	Разработва се от малка група в познатите условия на собствената фирма	Прости системи за управление на запаси или производствени процеси	$ЧМ = 2.4 \times ХРПК^{1.05}$
Полунезависим	Има междинно положение между разпространен и въраген тип	Системи за обработка на съобщения, нови операционни системи	$ЧМ = 3.0 \times ХРПК^{1.12}$
Въраген	Софтуерът работи свързан с апаратура, друг софтуер и изчислителни процеси	Авиационни или радиоелектронни системи	$ЧМ = 3.6 \times ХРПК^{1.20}$

б) Отчитане на допълнителни фактори във формулата за изчисляване на трудоемкостта и създаване на нови COSOMO модели

- Следващото усъвършенстване е свързано с установяването на факта, че все пак **редовете първичен код не биха могли да са единственият параметър** на установената формула.
- Създават се по-сложни COSOMO модели:
 - *междинен (COSOMO intermediate) и*
 - *детайлен (COSOMO detailed),*в които се отчитат допълнителни атрибути, със съответни рейтинги.

Оценявани атрибути

Атрибути на продукта

- Изисквана надеждност
- Размер на базата данни
- Сложност на продукта

Атрибути на компютъра

- Ограничение по бързина
- Ограничение по оперативна памет
- Изменяемост на виртуалната машина
- Цикъл на обръщение към компютъра

Атрибути на изпълнителя

- Квалификация на проектантите
- Опит в приложната област
- Квалификация на програмистите
- Опит от работа с компютъра
- Опит с езика за програмиране

Атрибути на проекта

- Прилагане на съвременни методи
- Ползване инструментални средства
- Ограничения в срока на разработка

**Оценявани
атрибути и
възможните
им **рейтинги****

Оценявани атрибути	Много нисък	Нисък	Нормален	Висок	Много висок	Свърх - висок
Атрибути на продукта						
Изисквана надеждност	0.75	0,88	1,00	1,15	1.40	
Размер на базата данни		0,94	1,00	1,08	1,16	
Сложност на продукта	0,70	0,85	1,00	1,15	1,30	1,65
Атрибути на компютъра						
Ограничение по бързина			1,00	1,11	1,30	1,65
Ограничение по оперативна памет			1,00	1,06	1,21	1,65
Изменяемост на виртуалната машина		0,87	1,00	1,15	1,30	
Цикъл на обръщение към компютъра		0,87	1,00	1,07	1,15	
Атрибути на изпълнителя						
Квалификация на проектантите	1,46	1,19	1,00	0,86	0,71	
Опит в приложната област	1,29	1,13	1,00	0,91	0,82	
Квалификация на програмистите	1,42	1,17	1,00	0,86	0,70	
Опит от работа с компютъра	1,21	1,10	1,00	0,90		
Опит с езика за програмиране	1,14	1.07	1,00	0,95		
Атрибути на проекта						
Прилагане на съвременни методи	1,24	1,10	1,00	0,91	0,82	
Ползване инструментални средства	1,24	1,10	1,00	0,91	0,83	
Ограничения в срока на разработка	1,23	1,08	1.00	1,04	1.10	

- При извършването на оценката експертите следва да се ръководят от тази таблица и за конкретния проект трябва да определят **за всеки атрибут, съответния рейтинг.**
- Получените стойности се заместват в формулата, която вече има по-сложен вид:

$$ЧМ = \kappa \times ХРПК^p \times A_1 \times A_2 \dots \times A_{15},$$
където ЧМ и ХРПК са вече познатите величини,
- κ (**2.4, 3.0, 3.6**) и p (**1.05, 1.12, 1.20**) са коефициенти, които са различни за различните типове софтуер, както се вижда и от табл.2 (слайд 23).

- $A_1, A_2 \dots, A_{15}$ са рейтингите на атрибутите (от слайд 26): те се умножават.
- Разликата в междинния и детайлния модел е в начина на получаване на крайната оценка.
- При детайлния вариант се прилагат **различни коригиращи коефициенти (рейтинги)**:
 - ✓ **за всяка фаза** от производствения процес, тоест има различна таблица, с рейтинги на атрибутите, всяка фаза.
 - ✓ започва се с оценка на **отделните модули**, от тях се получават оценки за **подсистемите** и от тях - за **цялостния продукт**.
- При междинния вариант нивото модул се игнорира и таблицата с атрибути и рейтинги е една единствена.

3.5. Критика на „редове първичен код”

Това в основната критика към СОСОМО, в следните 2 аспекта:

- 1. !!!Няма нито стандарт, нито единно виждане за това, какво е „ред първичен код”?**
- 2. Много е трудно, дори за експерти с голям опит, да предскажат достатъчно точно, в ранен етап на разработването броя на “редовете първичен код”.**

**Редове
първичен
код
(проблеми)**

```
graph TD; A((физически или логически редове?)) --- B((Редове първичен код (проблеми))); B --- C((Какви типове оператори се броят?)); C --- D((Как се отчита повторно използваният код?)); D --- E((Как се броят редовете първичен код, ако са писани на повече от един език?)); E --- F((Да се включва ли, при броенето на редовете първичен код, временно поставен в програмата код?)); F --- B;
```

*физически или
логически редове?*

*Какви типове
оператори се броят?*

*Как се отчита
повторно
използваният код?*

*Как се броят редовете
първичен код, ако са
писани на повече от
един език?*

*Да се включва ли,
при броенето на
редовете първичен
код, временно
поставен в
програмата код?*

Във връзка с редовете първичен код се изтъкват следните проблеми:

- а) **Какво всъщност да се разглежда — *физически или логически редове*.**
- **Физическият край** на даден ред се получава с натискането на клавиша **Enter**, което води до генериране на съответни служебни символи за край на реда.
 - **Логическият край** е някакъв ограничител, зависещ от конкретния език за програмиране, **например двоеточие**, запетая или друг точно определен знак. Езици, които позволяват написването на няколко оператора на един ред, могат да дадат неколнократна разлика при двата вида броене. Същото може да се случи в обратна посока и когато (за прегледност или по други причини) един логически оператор се разполага върху 2 или повече реда.
 - Изследване в САЩ е показало, че:
 - 35% от ръководителите на проекти броят физическите редове,
 - 15% броят логическите, а останалите:
 - 50% - въобще не смятат за уместно да броят редовете първичен код.

б) Какви типове редове първичен код (т.е какви типове оператори) да се разглеждат? Почти всеки процедурен език включва 4 типа първични редове:

- ***изпълними оператори***, чрез които се задават различни операции (логически, аритметични, вход/изход и др.);
- ***декларации на данни***, чрез които се дефинират различните типове данни;
- ***коментари***, които дават разясняваща информация;
- ***празни редове***, които се ползват за повишаване прегледността на програмата.

- Установено е, че в дадено, типично бизнес-приложение, около 40% от общия брой оператори за изпълними, 35% са декларации на данни, 15% са коментари и 10% — празни редове.
- При системния софтуер (например операционни системи) 45% са изпълними оператори, 30% са декларации на данни и отново 15% са коментари и 10% празни редове.
- Правени са изследвания сред създателите на софтуер и отново е установено различие при отчитането:
 - **10% броят само изпълнимите редове,**
 - **20 % броят изпълнимите редове и декларациите на данни,**
 - **15% добавят коментарите, а**
 - **5% броят дори и празните редове.**
 - **50% въобще не броят редовете първичен код.**

в) Как се отчита *повторно използваният код*?

- Изследванията сочат, че един програмист на по-старите процедурни езици за програмиране, средно копира от стари програми между **20 и 30% код**, в новите си програми.
- За обектно ориентирани езици, като C++ и др. - този процент достига **до 50%**.
- Има отделни софтуерни фирми, които са организирали специални библиотеки от модули за повторно използване и там въпросният процент достига дори **75%**.

При това положение спорът се върти около това как да се брой даден повторно използван модул:

- да се брой при всяко негово появяване;
- да се брой само веднъж, независимо от броя появявания;
- въобще да не се брой, доколкото този модул не е разработван за оценявания проект.
- Изследвания в САЩ показват, че:
 - първата алтернатива се прилага от 25% от професионалистите,
 - втората — от 20%,
 - третата — от останалите, т.е — въобще не се брой.

г) **Как се броят редовете първичен код, ако са писани на повече от един език? Тук също няма единство!**

- Има данни, че около 1/3 от софтуера в САЩ е написан на повече от един език за програмиране.

д) **Да се включва ли, при броенето на редовете първичен код , временно поставен в програмата код, код от типа макрос, код, свързан с управлението на заданието в рамките на операционната система. По този въпрос също няма единство!**

- Независимо от критиката по посочените 5 пункта, моделът **COSOMO продължава да има своите привърженици и да се прилага.**
- Причината:
 - невъзможността, поне за момента, да се предложи по-адекватен метод и модел.
 - огромните усилия, необходими за създаването, валидирането и прилагането на **НОВИ** методи за оценка.

COCOMO 2.0

- Развитието на COCOMO е COCOMO 2.0.
- Методът допълва COCOMO, позволявайки определяне на размера на проекта, както чрез:
 - ☐ „брой редове първичен код“, така и чрез
 - ☐ „метода на функционалните точки“ (или „обектни точки“).
- На <http://csse.usc.edu/tools/COCOMOII.php> е даден калкулатор, базиран на метода COCOMO 2.0.

COCOMO II - Constructive Cost Model

Model(s)

COCOMO

Monte Carlo Risk

Off

Auto Calculate

Off

Software Size

Sizing Method

Source Lines of Code

SLOC

% Design Modified

% Code Modified

% Integration Required

Assessment and Assimilation (0% - 8%)

Software Understanding (0% - 50%)

Unfamiliarity (0-1)

New

Reused

0

0

Modified

Software Scale Drivers

Precedentedness

Nominal

Architecture / Risk Resolution

Nominal

Process Maturity

Nominal

Development Flexibility

Nominal

Team Cohesion

Nominal

Software Cost Drivers

Product

Required Software Reliability

Nominal

Data Base Size

Nominal

Product Complexity

Nominal

Developed for Reusability

Nominal

Documentation Match to Lifecycle Needs

Nominal

Personnel

Analyst Capability

Nominal

Programmer Capability

Nominal

Personnel Continuity

Nominal

Application Experience

Nominal

Platform Experience

Nominal

Language and Toolset Experience

Nominal

Platform

Time Constraint

Nominal

Storage Constraint

Nominal

Platform Volatility

Nominal

Project

Use of Software Tools

Nominal

Multisite Development

Nominal

Required Development Schedule

Nominal

Maintenance

Off

Software Labor Rates

Cost per Person-Month (Dollars)

Calculate

4. Метод на функционалните точки (МФТ)

4.1. История и мотиви

- Методът на функционалните точки е предложен през още през 1979 г. от Олбрихт.
- Следва развитие на метода: Олбрихт, в съавторство с Гафмей, както и много други учени работят усилено и продължително върху усъвършенстването му, практическото му приложение, сравнението му с други модели и методи.
- Основният мотив за създаване са недостатъците на фактора брой редове първичен код, който е основата на няколко модела, сред които и COSOMO.

- Като най-съществени негови недостатъци вече изтъкнахме:
 - трудната ранна определяемост на броя редове първичен код (LOC) и
 - липсата на единно виждане за LOC.
- Поставяйки си за цел да намери по-добър фактор в това отношение (ясен и лесен за определяне в ранен стадий на производствения процес), Олбрихт достига до понятието „**функционална точка**“ (ФТ).

- На основата на този базов обект „функционална точка“, Олбрихт изгражда метод , който по-късно се осъвършенства.
- Основна идея на метода е, че:
 - **усилията за разработването на даден софтуер** (а следователно и цената му) се определят от неговата функционалност. Последната може да бъде измерена на основата на въпросните **функционални точки**.

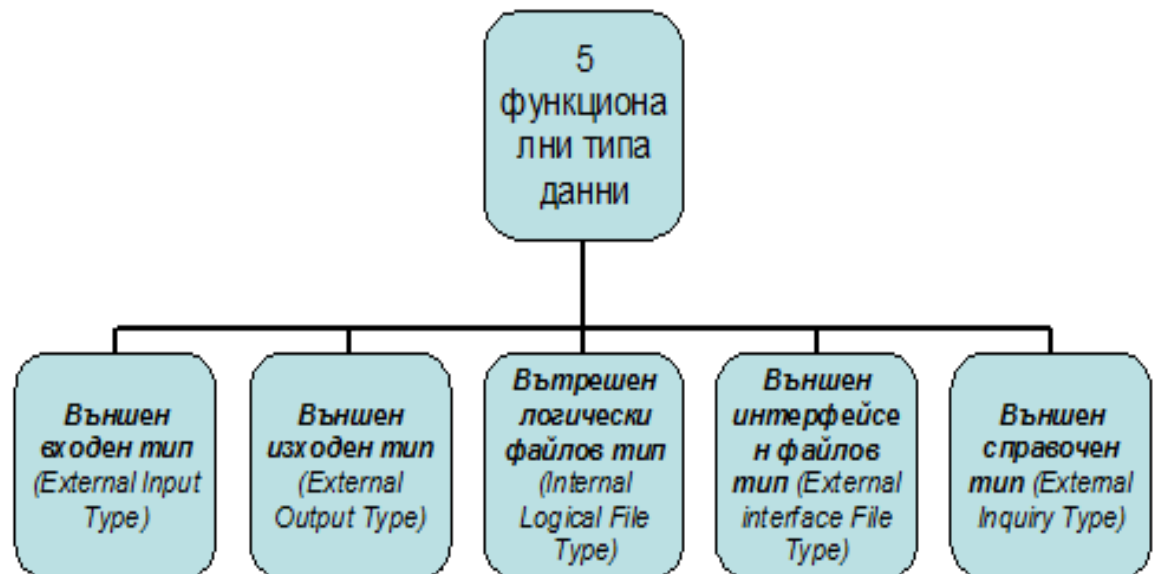
Предимства на функционалните точки (**ФТ**) пред брой редове първичен код (**LOC**):

- За разлика от LOC, ФТ (респ. техният брой и вид) могат да бъдат определени с помощта само на някои първоначални проектни документи, като спецификацията за изискванията (SRS), което означава - ранно получаване на желаните оценки.

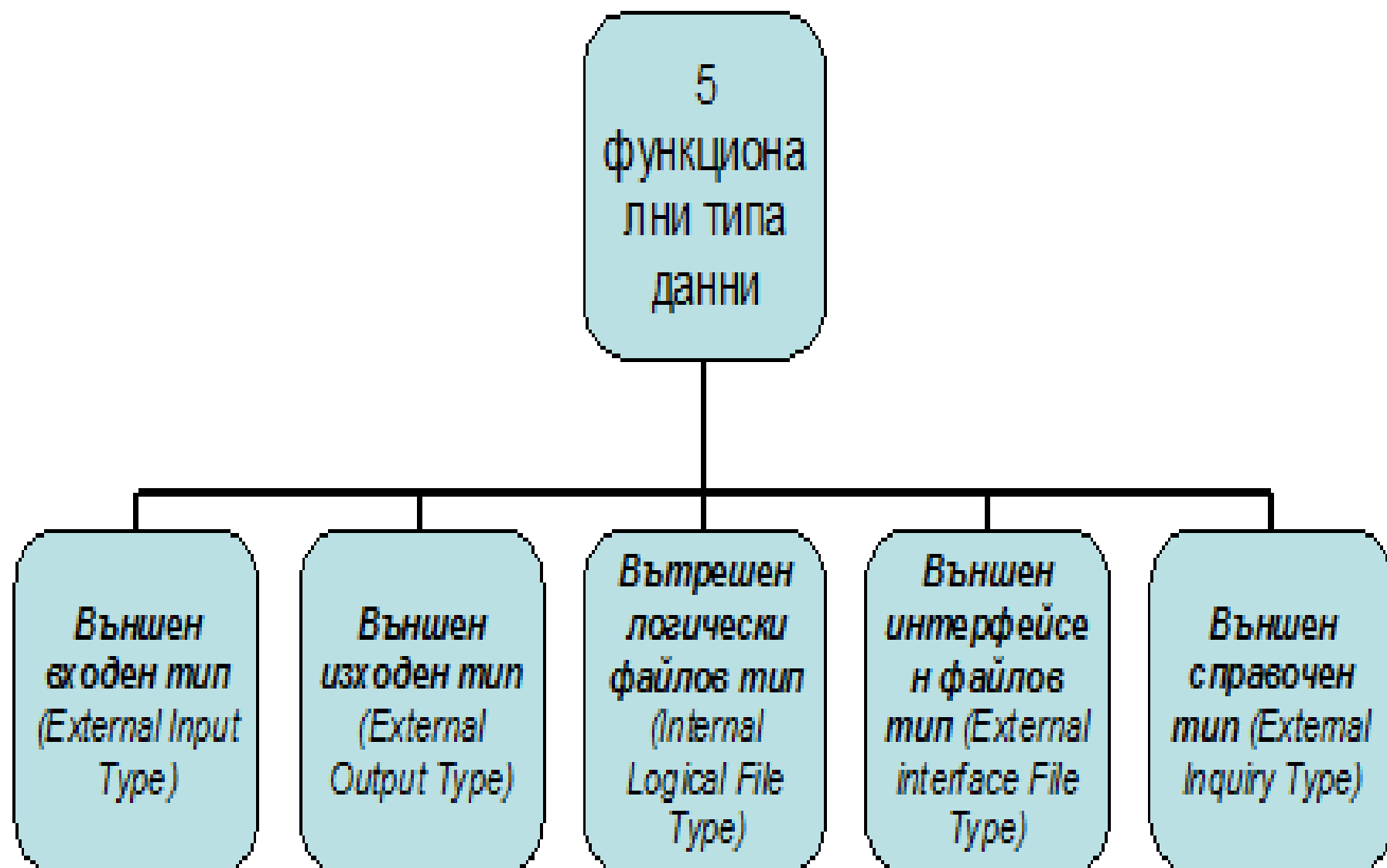
4.2. Същност на метода

- Основава на определянето, за всеки софтуерен продукт, на **5** непресичащи се **функционални типа данни** (функционални точки), всеки тип - с **3** нива на сложност:

- просто,
- средно,
- сложно.



Типовете ФТ са следните:



Външен входен тип (External Input Type)

- ***Външен входен тип (External Input Type)*** данни - такъв е **всеки потребителски входен** управляващ **поток** или поток от данни.
- Типичен пример за този тип са **входните екрани**. Въвежданите чрез тях данни "навлизат" в приложението и променят един или повече вътрешни логически типа.

.

Външен входен тип (External Input Type)

- Този тип може да бъде:
 - **прост**, когато съдържа малък брой входни типове данни и когато тези данни се отразяват върху малък брой вътрешни логически типове;
 - **междинен**, когато не може да се определи еднозначно, нито като прост, нито като сложен;
 - **сложен**, когато съдържа значителен брой входни типове данни и когато тези данни се отразяват върху голям брой вътрешни логически типове; при това проектирането на външния входен тип изисква значителни допълнителни усилия, тъй като трябва да се вземат предвид проблемите на човеко-машинния интерфейс.

Външен изходен тип (External Output Type)

- ***Външен изходен тип (External Output Type)*** — такъв е всеки потребителски изходен управляващ поток или поток от данни, който напуска външната граница на измерваното приложение.
- Като примери могат да се посочат **различни видове съобщения до потребителя или изходни отчети**.
- Класифицирането на този тип на трите нива на сложност е аналогично на предходния тип.

Вътрешен логически файлов тип ***(Internal Logical File Type)***

- Такава е всяка съществена група от потребителски данни или управляваща информация.
- Типичен пример: всяка логическа група от данни, обособена като такава поради потребителска представа и която се генерира, използва и поддържа от приложението.

- Класифицира се на три нива по следния начин:
 - **прост** - **малко типове записи** (н.р, таблици, малък брой колони), **малък брой типове елементи от данни** (н.р, малко на брой таблици в БД), не се изискват усилия за постигане на ефективност или за съхраняване и възстановяване;
 - **междинен** - типът не може еднозначно да се определи като прост или сложен;
 - **сложен** - голям брой типове записи и типове елементи от данни, изискват се специални грижи за осигуряване на по-висока ефективност, както и за процедури по запазване и възстановяване.

Външен интерфейсен файлов тип (External interface File Type)

- ***Външен интерфейсен файлов тип (External interface File Type)*** — това е файл, който се предава или се ползва съвместно от две или повече приложения.
- За да се определят трите нива на сложност, се ползват дефинициите за вътрешния логически файлов тип.

Външен справочен тип (External Inquiry Type)

- ***Външен справочен тип (External Inquiry Type)*** — всяка комбинация вход/изход, при която входът предизвиква незабавен изходен резултат.
- Такива комбинации се реализират например в дадена информационна система, когато потребителят формулира и въведе заявка и получи от приложението съответния отговор.

- Нивото на сложност на **външния справочен тип** (External Inquiry Type) се определя по следния начин:
 1. **Определяне сложността на входната част** (на външния справочен тип): класифицира се (прост, междунен и сложен) съгласно определенията за външния входен тип.
 2. **Определяне сложността на изходната част** (на външния справочен тип): класифицира се (прост, междунен и сложен) съгласно определенията за външния изходен тип;
 3. **Определяне сложността на външния справочен тип** - **равна е на по-голямата сложност, от така определените две сложности.**

4.3. Процедура по пресмятането

1. Преброяват се елементите на всеки от упоменатите 5 функционални типа (н.р нека имаме 10 външни входни типа).
2. След това за всеки елемент от всеки функционален тип се определя нивото на сложност съгласно дадените вече определения (за 10-те външни входни типа нека имаме, н.р: 3 – с просто ниво на сложност , 4 – с междинно, 6 – със сложно).
3. С така получените данни се попълва следната таблица:

4.3. Процедура по пресмятането

Тип	Наименование	прост	междинен	сложен	Общо
П	Външен входен	X 3	X 4	X 6	...
ОТ	Външен изходен	X 4	X 5	X 7	...
FT	Вътрешен логически файлове	X 7	X 10	X 15	...
EI	Външен интерфейсен файлове	X 5	X 7	X 10	...
QT	Външен справочен	X 3	X 4	X 6	...
FC	Общо ненастроени ф.точки				...

- Обозначението „X a" в трите колони за сложностите означава, че получената за тази клетка бройка трябва да се умножи по **теглото „a"**. Точките в колоната „Общо" означават, че на това място следва да се попълни сумата от получените в предходните 3 колони числа.
- Като се сумират 5-те числа от най-дясната колона, се получава общият брой **ненастроени функционални точки FC** за изследваното приложение (**FC** - т.е **Unadjusted Function Points**).

- Следващата стъпка хронологично е предложена по-късно. Тя е предизвикана от осъзнатата от авторите необходимост от съществени уточнения и “**донастройки на така получения брой FC**” (до получаване на **FP, т.е Adjusted Function Points, $FP = FC \times PCA$**).
- За донястройка на **FC**, се изчислява се един коригиращ коефициент **PCA (processing complexity adjustment)** по следния начин:
 - Разглеждат се изброените по-долу **14 характеристики** и за всяка от тях се определя степента и на влияние върху оценяваното приложение.

Самите *характеристики* са следните:

1. Данните и управляващата информация, използвани в приложението, се изпращат или получават по комуникационни линии.
2. В приложението има разпределена обработка на данни.
3. За приложението е важно достигането на висока ефективност.
4. Приложението ще се експлоатира върху силно натоварена операционна конфигурация — хардуер и софтуер.
5. Интензивността на транзакциите е висока.
6. Наличен е интерактивен режим на въвеждане на данни и на управляваща информация.
7. Цели се ефективност от гледище на потребителя.
8. Наличен е интерактивен режим на актуализирането на данните.
9. Логиката на обработките е сложна (многобройни логически и математически уравнения, необходимост от обработка на много изключения и др.).
10. Програмният код трябва да е повторно използваем (reusable).
11. Цели се лесно инсталиране.
12. Цели се лесна експлоатация (ефективна начална процедура, съхраняване, възстановяване и др.).
13. Приложението може да се ползва за разнообразни потребители.
14. Приложението е *гъвкаво* и лесно се модифицира.

- *Допустимите стойности* за всяка характеристика са от **0 до 5** и имат следния смисъл:

**0— не е налична или не влияе
въобще**

1— незначително влияние

2— умерено влияние

3— средно влияние

4— значително влияние

5— силно влияние.

Изчисляване на FP - краен (настроен) брой функционални точки

- Определя се величината **PC (processing complexity)** като сума от 14-те определени стойности. Очевидно тя е в интервала **[0,70]**.
- На нейна основа се определя коригиращият коефициент **PCA** (processing complexity adjustment), по формулата:
 $PCA = 0.65 + (0.01 \times PC)$
- Ясно е, че PCA ще бъде в интервала **[0.65, 1.35]**.
- Настроеният брой функционални точки **FP** (т.е **Adjusted Function Points**), се получава по формулата:
 $FP = FC \times PCA$
- Вижда се, че коригиращият коефициент PCA всъщност коригира първоначалния резултат **FC** с **+/- 35%**.

4.4. Използване на модела

1. За изчисляване на **цената на разработване на даден софтуерен продукт:**

- При наличието на достатъчно статистически данни не е трудно да се определи цената на създаване на една функционална точка.
- Оттук лесно може да се пресметне и очакваната цена на разработване на даден софтуерен продукт.

4.4. Използване на модела

2. **За формулиране на изводи**, базирани на натрупани статистически данни, по отношение на броя функционалните точки, използвани от:
- Организации (какъв е приблизителният брой функционални точки, притежавани от определен тип организация в САЩ?).
 - Професии (какво количество функционални точки използват избрани групи от потребители в САЩ?).

Тип организация (фирма)	Брой функц. точки
Малка местна банка	125 000
Средна търговска банка	350 000
Голяма международна банка	450 000
Средна животозастрахователна компания	400 000
Голяма животозастрахователна компания	550 000
Голяма телефонна оперативна компания	450 000
Голяма телефонна производствена компания	600 000
Средна производствена компания	200 000
Голяма производствена компания	375 000
Голям производител на компютри	1 650 000

Табл. 10.5. Брой функционални точки по типове организации

Тип професия	Брой използвани функционални точки
Служител в туристическа агенция	35 000
Служител по резервациите в авиолиния	30 000
Аерокосмически инженер	25 000
Електроинженер	25 000
Телекомуникационен инженер	20 000
Банков служител по кредитите	15 000
Софтуерист	15 000
Машинен инженер	12 500
Ликвидатор в застрахователна компания	5 000

Табл. 10.6. Брой функционални точки по професии

5. Други модели

5.1.Doty

- Този модел наподобява COSOMO. Основава се на 2 формули:

$$\text{ЧМ} = 5.288 \times \text{ХРПК}^{1047}, \text{ за } \text{ХРПК} < 10$$

$$\text{ЧМ} = 2.060 \times \text{ХРПК}^{1047} \times f_1 \times f_2 \times \dots \times f_{14}, \text{ за } \text{ХРПК} \geq 10,$$

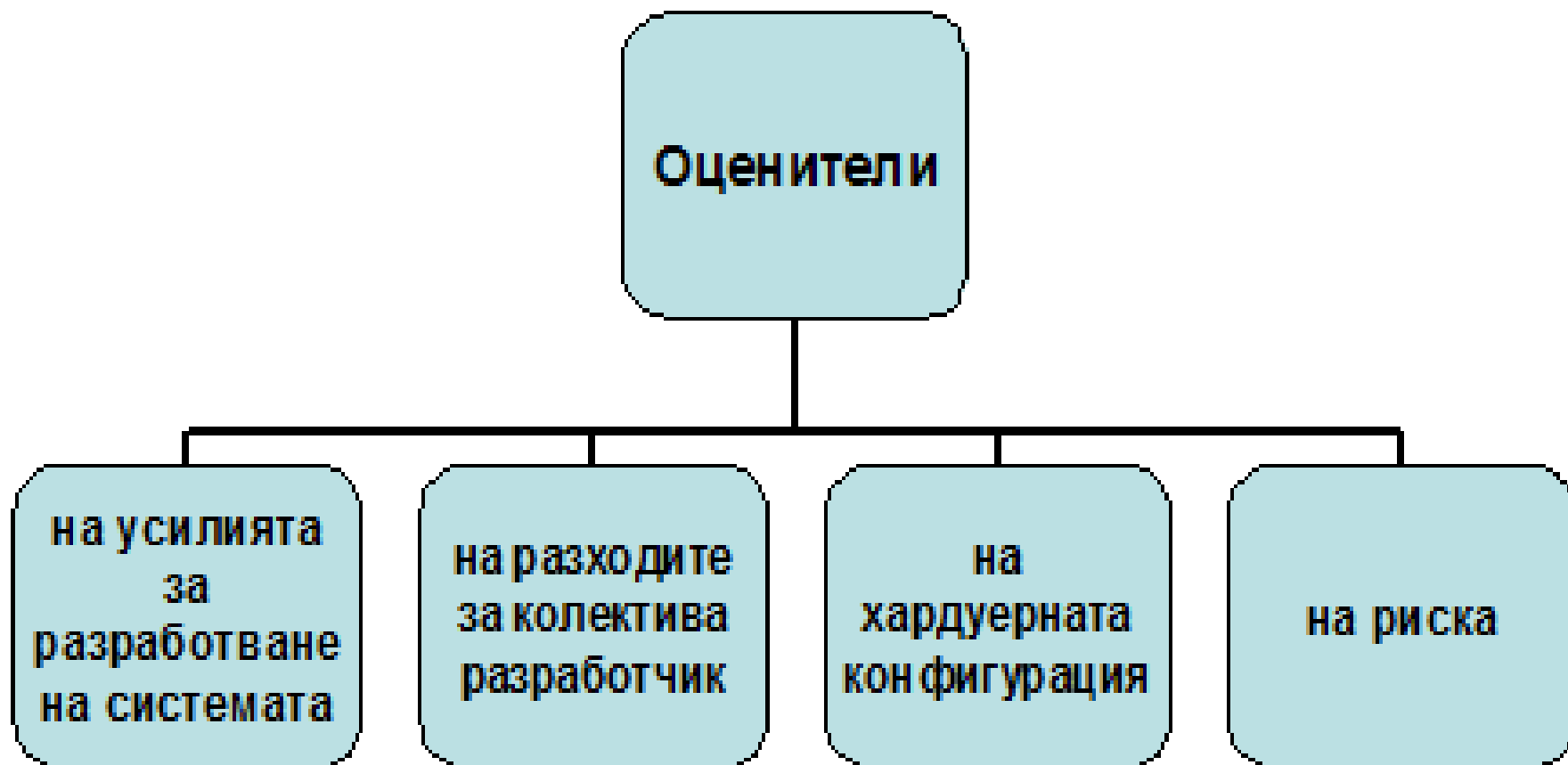
- където обозначенията имат същия смисъл, както при **COSOMO**, а f_i са подобни атрибути (като на **COSOMO**), но само с по 2 възможни стойности.

5.2. **SPQR** (Software productivity, quality, and reliability [1,2, 6])

- Създаден от Кейпърс Джоунс (1985), отново е инспириран от COSOMO.
- Основава се на **20 добре дефинирани и 25 недостатъчно добре дефинирани фактора**, определящи цената на разработване на софтуера и на производителността. Определят се на база дискусия с потребителите (от потребителя се изисква да отговори на **100 въпроса**, свързани със софтуерния проект).
- Авторът на модела Кейпърс Джоунс твърди, че с 90% вероятност моделът дава оценка в рамките на точност от **+/- 15%**.
- Доведен е до търговски продукт, но не е достатъчно добре документиран.

5.3. *ESTIMACS*

- Този модел (автор Рубин) е съсредоточен преди всичко на интервюиране на разработчиците и оценяване на:



- 1. Оценител на усилията за разработване на системата.** Основава се на отговорите на 25 въпроса относно разработвания софтуер, средата на разработване и др, базира се на натрупан опит от подобни разработки.
- 2. Оценител на разходите за колектива разработчик.** Входни данни са получената по т. 1 оценка, данни за квалификацията и производителността на разработчиците, разбивка на заплатите по нива. И тук се ползват исторически данни.
- 3. Оценител на хардуерната конфигурация.** На входа се подават данни за изискванията на софтуерния продукт към операционната среда, очаквания брой и тип на транзакциите, а като изход се получава оценка за необходимата хардуерна конфигурация.

4. Оценител на риска. На основата на отговори на 60 въпроса относно размера на проекта, структурата му и използваната технология се изчисляват характеристики на риска.

Претенциите на ***ESTIMACS*** са също за точност в границите на 15%.

5.4. *BANG*

- Авторът на този модел е една от най-известните фигури в областта на софтуерните технологии — Де Марко [1,2,6]. Методът е от типа на функционалните точки, но се знае, че е разработен, независимо от метода на Олбрихт.
- Докато моделът и методът на функционалните точки на Олбрихт е насочен най-вече към информационни системи и бизнес-приложения, целта на BANG е **системният софтуер и този за научни приложения**.

- BANG се базира се на установяване на значително по-широко множество от фактори (в сравнение с модела на Олбрихт), което го прави по трудно приложим:
 - Функционални примитиви (функционална единица, която не може повече да бъде декомпозирана).
 - Модифицирани функционални примитиви.
 - Елементи от данни.
 - Входни елементи от данни.
 - Изходни елементи от данни.
 - Елементи от данни в паметта.
 - Обекти (objects или entities).
 - Връзки (relationships) и др.

6. Оценяване на разходи при ОО подход за разработване

- Jones отбелязва в [7] (Jones C., The Economics of Object-Oriented Software), че при ОО подход за разработване на софтуер е неподходящо да се прилагат модели за оценка на разходите, базирани на броя първични редове код [1],[2],[7],[6].
- Едновременно с това се твърди, че **методът на функционалните точки** е приемливо приложим.

- Действително, всички специално разработени модели за оценка при ОО разработка са базирани на функционалните точки.

Известни модели:

- Модел и метод MOOSE (Metric for object-oriented system environment), публикуван през 1993 от Кемпра (Kemerer).
- Модел на Снийд.
- Модел на Боем , базиран на **COCOMO 2.0**, и т.н ***обектни точки (object points)***.

Въпроси

1. Обосновете необходимостта от оценка на разходите за разработка на софтуер. Какви критерии следва да удовлетворява всеки модел (метод) за установяване цената на разработване на софтуерни продукти?
2. Каква е целта на метода COSOMO, коментирайте неговата същност: оценъчни елементи, уравнения за пресмятане, ограничения и форми на осъвършенстване?
3. Коя е основната критика към метода COSOMO? Какво отличава COSOMO 2.0 от COSOMO?
4. Метод на функционалните точки (МФТ): същност, типове ФТ, предназначение. Предимства на функционалните точки пред брой редове първичен код. Процедура по пресмятането на ФТ.
5. Методи Doty, SPQR, BANG и ESTIMACS: същност и оценъчни елементи. Какви са особеностите при оценяване на разходите при ОО подход за разработване на софтуер?

Основна литература

1. Нели Манева, Аврам Ескенази, Софтуерни технологии. Анубис София 2001
2. Нели Манева, Аврам Ескенази, Софтуерни технологии. КЛМН, София 2006
3. Sommerville: Software Engineering , 9. ed. Addison-Wesley, 2011,
4. Barry Boehm, Chris Abts, Software Development Cost Estimation Approaches – A Survey,
<http://csse.usc.edu/TECHRPTS/2000/usccse2000-505/usccse2000-505.pdf>
5. Boehm B.W., Software Engineering Economics, Prentice Hall, Englewood Cliffs, N.J., 1981.
6. HOW SOFTWARE ESTIMATION TOOLS WORK, 2005,
<http://www.compaid.com/caiinternet/ezone/capers-estimation.pdf>
7. Jones C., The Economics of Object-Oriented Software